What Google Knows: Measuring Telemetry and Identifier Exposure Across Privacy-Focused Android Operating Systems

Bruno Mirčevski
Faculty of Computer Science, Bialystok University of Technology
Bialystok, Poland

Krzysztof Jurczuk
Faculty of Computer Science, Bialystok University of Technology
Bialystok, Poland

Abstract

This paper presents a preliminary comparative analysis of user privacy in Android operating systems through network traffic inspection. Using a controlled man-in-the-middle (MITM) setup and traffic decryption techniques, six Android systems were evaluated: stock Android, GrapheneOS, iodéOS, and three variants of LineageOS (vanilla, Gapps, microG). While prior work has focused on applications or stock Android across device manufacturers, this study investigated alternative Android distributions designed with privacy as a primary objective. We examined outbound connections, transmitted identifiers, and network traffic patterns to assess the impact of integrated Google services on user privacy.

Keywords

android, privacy, network traffic

1 Introduction

Android is the dominant mobile operating system and acts as a primary gateway to the digital world for billions of users. It is crucial for the core operating system to be a reliable, stable, and trustworthy platform that allows users to run applications and services of their choice. While third-party application privacy has been widely studied [1, 6, 7, 9], less attention is typically paid to the operating system layer and to privileged service frameworks that silently enable additional functionality.

While the Android core is the Android Open Source Project (AOSP), most consumer devices include GMS (Google Mobile Services) layered on top. They are implemented as privileged system applications to provide baseline functionality. Application developers commonly assume their presence by default. This design tightly couples many functions into a single service stack. Providing useful capabilities may implicitly enable others that a user would not choose, such as advertising identifiers, telemetry, and behavioral tracking. Because Google's business model is strongly advertising-driven, many users may be unwilling to entrust this provider with broad access to their devices and data.

In principle, users should be able to choose which services they rely on and have confidence that these choices are enforced by the platform's permission and isolation mechanisms. This demand for transparency and controllability motivates alternative approaches such as microG[1] and GrapheneOS sandboxed Google Play[2]. These solutions aim to preserve device functionality and application compatibility while reducing privilege and improving user control over Google-related components.

Privacy-focused Android operating systems aim to reduce data exposure at the OS level. They achieve this by limiting preinstalled privileged components, tightening default settings, and providing more transparent controls. We focus on three representative approaches. GrapheneOS[3] prioritizes a hardened security model and strong isolation. Google components are not included by default, but can be installed in a sandboxed mode as ordinary applications. LineageOS[4] provides a lightweight, broadly supported AOSP-based system with minimal preinstalled software. It can be deployed in three configurations: without Google services, with Gapps[5] (proprietary Google Play Services), or with microG[6]. Finally, iodéOS[7] is a privacy-oriented distribution that combines a de-Googled baseline with additional privacy features and preconfigured app ecosystem choices, shipping with microG.

Prior work has established that Android privacy risks arise not only from user-installed applications, but also from the operating system and its preinstalled components [2–5, 8]. Official Android builds transmit substantial data to OS vendors and third parties, including telemetry, location, app activity, and persistent identifiers. Google serves as the dominant driver of this background tracking ecosystem. Prior literature shows that removing or replacing these components with open-source variants drastically reduces unsolicited communication [4]. However, alternative distributions remain underexplored in reproducible measurement studies. This motivates our systematic comparison of stock and privacy-oriented systems.

Our main contributions are as follows: 1) we present a comparative network-level privacy analysis of six Android configurations across diverse Google integration models; 2) we provide the first systematic empirical comparison between privileged GMS, sandboxed Google Play Services, and microG under identical conditions; 3) we design a controlled, real-device traffic interception and TLS decryption measurement methodology tailored for Android 16; 4) we quantify how alternative distributions reduce background data and identifier exposure while preserving core functionality.

2 Experiment design

We used a single Google Pixel 6a device (codename *bluejay*) to evaluate the studied Android distributions. All tested builds are based on Android 16 and were obtained in December 2025: Stock Android (BP4A.251205.006); LineageOS (lineage-23.0-20251206); LineageOS for microG (lineage-23.0-20251203-microG); GrapheneOS (2025121200); iodéOS (7.0-20251129-bluejay).

[1] https://github.com/microg/GmsCore/wiki
[2] https://grapheneos.org/usage#sandboxed-google-play
[3] https://grapheneos.org/
[4] https://lineageos.org/
[5] https://mindthegapps.com/
[6] https://lineage.microg.org/
[7] https://iode.tech/iodeos/

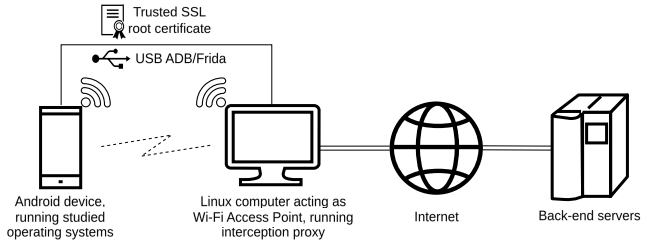

Figure 1: Measurement setup. The Android device connects to a Wi-Fi access point hosted on a Linux computer that runs traffic interception and capture software. A custom system certificate is installed on the phone using root access. A USB connection is used for ADB/Frida communication.

Network traffic analysis on Android is challenging due to enforced encryption, certificate pinning, and anti-tamper mechanisms. Traffic was captured using a controlled MITM gateway with a Linux host acting as a Wi-Fi router, redirecting HTTP(S) traffic through an intercepting proxy. QUIC over UDP/443 was blocked to force TCP-based TLS. The proxy CA was installed as a system-trusted certificate. We performed runtime certificate unpinning via Frida using scripts from HTTP Toolkit[8]. Full packet captures were recorded and compared across distributions. Where needed, we used publicly available Protocol Buffer definitions from microG's source code[9] to decode payloads. We acknowledge that we were not able to capture and analyze all traffic. Instead, we focused on identifying systematic differences between Android distributions and general behavioral patterns. A preview of the measurement setup is shown in Fig. 1.

2.1 Measurement scenarios

To enable consistent and comparable measurements across Android distributions, four scenarios were defined. Scenarios were adapted to each system's capabilities, or skipped when not applicable. As a general principle, any required configuration step not available during initial setup was performed as soon as possible thereafter.

Scenario A1: Default setup with Google account. Typical user path. Google account sign-in is performed during setup and default recommendations are accepted (including prompted permissions).

Scenario A2: Minimal setup with Google account. This scenario extends *Scenario A1* by adjusting settings toward privacy while retaining Google account functionality. During and after initial setup, all optional consents and permissions are declined, and settings are adjusted to maximize privacy. For privacy-focused systems, the distinction between Scenario A1 and Scenario A2 is minimal; therefore, in this case we skipped Scenario A2.

Scenario B: Minimal setup without Google account. With maximum privacy in mind, avoiding Google account sign-in entirely. Only strictly required prompts are accepted. Google components remain present and enabled, but operate without an account. This scenario applies to the same systems as Scenario A1.

[8]https://github.com/httptoolkit/frida-interception-and-unpinning
[9]https://github.com/microg/GmsCore/

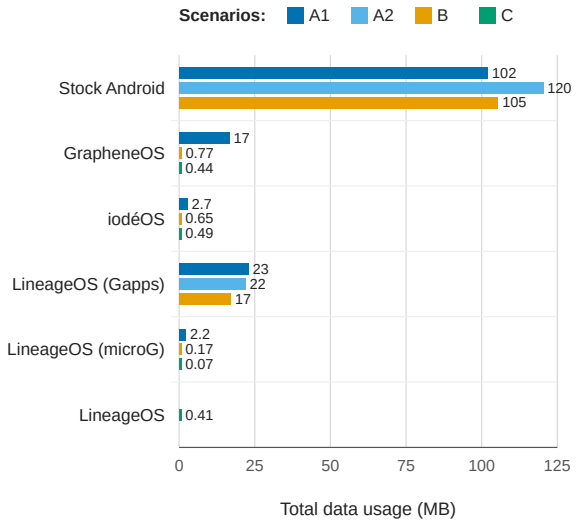

Figure 2: Total data usage aggregated across all measurement tasks. Scenario applicability varies by system.

Scenario C: Extremely minimal setup without Google components. Applies only to distributions that do not enable Google services by default. If microG is preinstalled, it remains disabled.

2.2 Measurement tasks

We performed the following tasks sequentially for each scenario. Network traffic was captured using Wireshark[10] for basic analysis.

Task 0: Initial setup. The device was configured according to the target scenario. Network traffic capture began with the device's first Internet connection and continued until 10 minutes after setup completion.

Task 1: System idle. The device was rebooted, then kept connected to the monitored network for at least 30 minutes. During this period, the device was unlocked three times and the application list was scrolled through. No applications were opened.

Task 2: Basic application interactions. Simple actions were performed: disable haptic feedback; toggle dark mode; check per-application battery usage; set an alarm; create a folder; create an on-device contact.

3 Results

Total data usage across all operating systems and scenarios, aggregated for all tasks, is presented in Figure 2. In all scenarios stock Android shows the highest data usage. Generally, configurations incorporating more Google services generate more traffic.

Figure 3 presents the domain-level distribution of network traffic. Stock Android contacts the most Google-related domains. iodéOS and LineageOS contact fewer servers overall. iodéOS also relies on some Mozilla infrastructure not present in other systems.

Table 1 shows the infrastructure dependencies for core system functions. LineageOS remains closest to stock Android, relying entirely on Google servers. iodéOS adopts an intermediate position,

[10]https://www.wireshark.org/

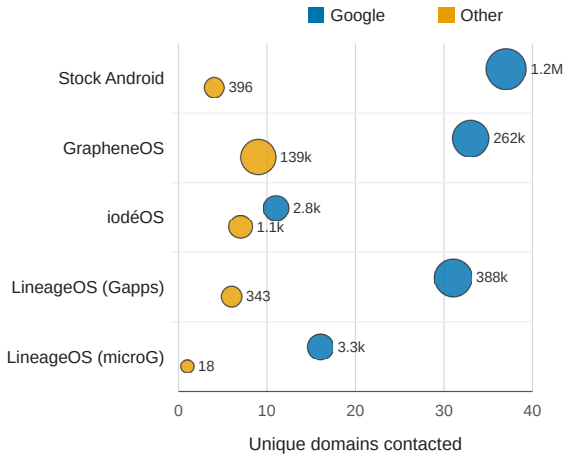

Figure 3: Domains contacted in Scenario A1 aggregated for all tasks, grouped by operating system and by Google-related and other endpoints. Bubble size represents the total number of packets exchanged with each group. Vanilla LineageOS was omitted due to lack of available scenarios

while GrapheneOS demonstrates the strongest de-Googling, substituting all three functions with its own or independently operated infrastructure.

Beyond aggregate traffic measurements, we were able to decrypt and analyze some payloads. A subset of these observations comes from earlier experiments performed on Android 14, because the Frida-based method was unsuccessful on Android 16.

Android's check-in mechanism transmits actual IMEI, MAC address, and hardware serial number to Google, while microG generates randomized, rotating identifiers (IMEI consistently prefixed with `35503104`, MAC addresses with OUI `b4:07:f9`). The Android ID field in microG requests is frequently empty or zeroed. The Google Advertising ID (ADID), formatted as a UUID, was observed in multiple requests on stock Android. This value remained constant across different requests, confirming its persistence as a cross-application tracking mechanism. After a manual reset in system settings, the ADID was transmitted as all zeros, indicating the identifier was actually cleared. We observed requests to *app-measurement.com* on systems with Google services present, containing application package names, installation sources, versions, and the system theme preference.

When enabled, microG performs network geolocation without Google infrastructure by querying a selected provider, in our case *api.position.xyz*. The request transmits nearby cell towers and returns approximate coordinates. We compared Google Play Store with Aurora Store (a popular application source in alternative Android distributions) by downloading the same application from each platform. Both stores retrieved identical packages from the same Google CDN infrastructure. However, the Play Store generated 106 requests, while Aurora Store produced 93.

Table 1: Domains observed for connectivity check, remote provisioning, and location assistance.

System	Connectivity check	Provisioning	Location
Stock	connectivitycheck.gstatic.com	–	supl.google.com, agnss.goog
Lineage	connectivitycheck.gstatic.com	remoteprovisioning.googleapis.com	supl.google.com, agnss.goog
iodéOS	captiveportal.kuketz.de	remoteprovisioning.googleapis.com	supl.google.com
Graphene	connectivitycheck.grapheneos.network	remoteprovisioning.grapheneos.org	gs-loc.apple.grapheneos.org

Table 2: Comparison of Google Takeout data across Android configurations.

System	Device IDs	Apps	Pixel telem.
Stock Android	IMEI, serial	Extensive	Full
Lineage (Gapps)	IMEI, serial	Extensive	Reduced
Lineage (microG)	–	–	–
GrapheneOS	Falsified serial	4 apps	–

In addition to network-level observations, Google Takeout[11] was used to download data associated with test accounts for different Google services implementations. For every logged-in system, a corresponding entry appeared in `devices.json`, including device model, OS version, country, language, registration timestamp, and last activity time. However, the depth of telemetry varied considerably across configurations, as summarized in Table 2.

4 Conclusion

Our evaluation demonstrates that alternative Android distributions significantly reduce unsolicited data transmissions, offering privacy protections that standard stock Android settings cannot replicate. Ultimately, the choice depends on the user's specific privacy and usability needs. It is a preliminary study and future research could extend the measurement period and conduct repeated trials to better isolate systematic behavior from transient network variations.

References

[1] Haojian Jin, Minyi Liu, et al. 2018. Why Are They Collecting My Data? Inferring the Purposes of Network Traffic in Mobile Apps. *Proc. ACM IMWUT* 2, 4, Article 173 (2018). doi:10.1145/3287051

[2] Douglas J. Leith. 2023. What Data Do the Google Dialer and Messages Apps on Android Send to Google?. In *SecureComm*. 549–568.

[3] Douglas J. Leith. 2025. Cookies, Identifiers and Other Data That Google Silently Stores on Android Handsets. https://www.scss.tcd.ie/doug.leith/pubs/cookies_identifiers_and_other_data.pdf

[4] Haoyu Liu, Paul Patras, and Douglas J. Leith. 2021. Android Mobile OS Snooping by Samsung, Xiaomi, Huawei and Realme Handsets. https://www.scss.tcd.ie/doug.leith/Android_privacy_report.pdf

[5] Haoyu Liu, Paul Patras, and Douglas J. Leith. 2023. On the data privacy practices of Android OEMs. *PLOS ONE* 18, 1 (2023), 1–15. doi:10.1371/journal.pone.0279942

[6] Abbas Razaghpanah, Rishab Nithyanand, et al. 2018. Apps, Trackers, Privacy, and Regulators. In *NDSS*. doi:10.14722/ndss.2018.23009

[7] Jingjing Ren, Martina Lindorfer, et al. 2018. Bug Fixes, Improvements, ... and Privacy Leaks. In *NDSS*.

[8] Douglas C. Schmidt. 2018. Google Data Collection. https://digitalcontentnext.org/wp-content/uploads/2018/08/DCN-Google-Data-Collection-Paper.pdf

[9] Zhaohua Wang, Zhenyu Li, et al. 2020. Exploring the Eastern Frontier: A First Look at Mobile App Tracking in China. In *PAM*. 314–328.

[11] https://takeout.google.com/